Script to validate points in ReSurvey database using RS data (S2 + SatEmb + Landsat)

K-fold cross validation of best models

Alicia Valdés

30 enero 2026

This R script is used to validate the points in the ReSurvey database using either RS indicators from S2 or Landsat (indices + phenology + canopy height), or bands from the Google Satellite Embedding dataset: https://developers.google.com/earth-engine/datasets/catalog/GOOGLE_SATELLITE_EMBEDDING_V1_ANNUAL#description Here, we perform a K-fold cross validation of the best models.

Load libraries

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.2.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(here)
```

```
## here() starts at C:/Users/jimenezalfaro/OneDrive - Universidad de Oviedo/IMIB/Analyses/MOTIVATE/MOTI
```

```r
library(parallel)
library(doParallel)
```

```
## Cargando paquete requerido: foreach
##
## Adjuntando el paquete: 'foreach'
##
## The following objects are masked from 'package:purrr':
##
##     accumulate, when
##
## Cargando paquete requerido: iterators
```

```r
library(moreparty)
```

```
## Cargando paquete requerido: party
## Cargando paquete requerido: grid
## Cargando paquete requerido: mvtnorm
## Cargando paquete requerido: modeltools
## Cargando paquete requerido: stats4
## Cargando paquete requerido: strucchange
## Cargando paquete requerido: zoo
##
## Adjuntando el paquete: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
##
## Cargando paquete requerido: sandwich
##
## Adjuntando el paquete: 'strucchange'
##
## The following object is masked from 'package:stringr':
##
##     boundary
##
##
## Adjuntando el paquete: 'party'
##
## The following object is masked from 'package:dplyr':
##
##     where
```

```r
library(caret)
```

```
## Cargando paquete requerido: lattice
##
## Adjuntando el paquete: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Adjuntando el paquete: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Adjuntando el paquete: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```r
library(rlang)
```

```
##
## Adjuntando el paquete: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##     flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##     flatten_raw, invoke, splice
```

```r
library(stringr)
library(beepr)
library(permimp)
library(purrr)
library(MLmetrics)
```

```
##
## Adjuntando el paquete: 'MLmetrics'
##
## The following objects are masked from 'package:caret':
##
##     MAE, RMSE
##
## The following object is masked from 'package:base':
##
##     Recall
```

Load functions

```r
knitr::purl("functions_20260122.Rmd", output = "functions_20260122.R")
```

```
## [1] "functions_20260122.R"
```

```r
source("functions_20260122.R")
```

Load previously created objects

```r
# Load best model Landsat (and prediction) - Accuracy = 0.730 (without k-fold)
model_Landsat <- get(load(here("objects", "RF_1500_20260128",
                               "rf_months_Landsat_refin_20_10_2x_model.Rdata")))
prediction_Landsat <- get(load(here("objects", "RF_1500_20260128",
                                    "rf_months_Landsat_refin_20_10_2x_prediction.Rdata")))
# Load filtered data Landsat
filtered_data_months_Landsat_with_probs_2x <- get(load(here("objects", "RF_1500_20260128",
                                        "filtered_data_months_Landsat_with_probs_2x.Rdata")))
# Load best model S2 or SatEmb (and prediction) - Accuracy = 0.7413 (without k-fold)
model_SatEmb <- get(load(here("objects", "RF_1500_20260128",
                              "rf_SatEmb_refin_20_10_2x_model.Rdata")))
prediction_SatEmb <- get (load(here("objects", "RF_1500_20260128",
                                    "rf_SatEmb_refin_20_10_2x_prediction.Rdata")))
# Load filtered data S2 or SatEmb
filtered_data_SatEmb_with_probs_2x <- get(load(here("objects", "RF_1500_20260128",
                                        "filtered_data_SatEmb_with_probs_2x.Rdata")))
```

```r
# Load cross-validation objects
cv_SatEmb <- get(load(here("objects", "RF_1500_cv", "cv_SatEmb.Rdata")))
cv_Landsat <- get(load(here("objects", "RF_1500_cv", "cv_Landsat.Rdata")))
```

Read data

```r
data_validation_SatEmb <- read_tsv(here(
  "data", "clean","final_RS_data_SatEmb_20251010.csv")) %>%
  rename(EUNISa_1_descr = EUNIS_1_d)
```

```
## Rows: 36552 Columns: 115
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr  (31): Country, Dataset, Dtofrcr, EUNIS_1_d, EUNIS_2_d, EUNIS_3_d, EUNIS...
## dbl  (69): PlotObservationID, A00, A01, A02, A03, A04, A05, A06, A07, A08, A...
## lgl  (14): EUNIS_4_d, EUNISa_4, EUNISb_1, EUNISb_4, EUNISb_4_, EUNISc, EUNIS...
## date  (1): date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
data_validation_Landsat <- read_tsv(here(
  "data", "clean","final_RS_data_bands_Landsat_all_20251021.csv"))
```

```
## Rows: 175039 Columns: 137
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr   (7): EUNISa_1, EUNISa_1_descr, EUNISa_2, EUNISa_2_descr, biogeo, unit,...
## dbl (130): PlotObservationID, EVI_auc_mar_oct, EVI_auc_slope, EVI_auc_thresh...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

No parsing issues!

```r
nrow(data_validation_Landsat)
```

```
## [1] 175039
```

```r
nrow(data_validation_SatEmb)
```

```
## [1] 36552
```

Filter out manipulated observations

Read list of PlotObservationIDs to keep

```r
PlotObservationID_list <- read_csv(
  here("data", "clean","PlotObservationIDs_filtered_QRST_20260122.csv"))
```

```
## Rows: 165385 Columns: 1
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (1): PlotObservationID
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Filter out data to keep only those PlotObservationIDs

```r
data_validation_SatEmb <- data_validation_SatEmb %>%
  filter(PlotObservationID %in% PlotObservationID_list$PlotObservationID)
data_validation_Landsat <- data_validation_Landsat %>%
  filter(PlotObservationID %in% PlotObservationID_list$PlotObservationID)
```

```r
nrow(data_validation_Landsat)
```

```
## [1] 148130
```

```r
nrow(data_validation_SatEmb)
```

```
## [1] 31359
```

Define lists of predictor vars

```r
# Excluding EVI because many values outside of range -1, 1
vars_RF_months_Landsat <- c(
  # Min values of all indices
  "NDVI_min", "NDMI_min", "NDWI_min", "SAVI_min",
  # Max values of NDMI and NDWI
  "NDMI_max", "NDWI_max",
  # AUC of NDVI, and SAVI between march and oct
  "NDVI_auc_mar_oct", "SAVI_auc_mar_oct",
  # Values of NDVI and SAVI at march, pos and oct
  "NDVI_avg_value_03", "NDVI_pos_value", "NDVI_avg_value_10",
  "SAVI_avg_value_03", "SAVI_pos_value", "SAVI_avg_value_10",
  # Differences pos-march in value and doy
  "NDVI_diff_pos_march_value", "SAVI_diff_pos_march_value",
  "NDVI_diff_pos_march_doy", "SAVI_diff_pos_march_doy",
  # Differences pos-oct in value and doy
  "NDVI_diff_pos_oct_value", "SAVI_diff_pos_oct_value",
  "NDVI_diff_pos_oct_doy", "SAVI_diff_pos_oct_doy",
  # Canopy height
  "canopy_height")
```

```r
vars_RF_SatEmb <- colnames(
  data_validation_SatEmb)[startsWith(colnames(data_validation_SatEmb), "A")]
```

Get filtered data

Landsat

```r
filter_data_Landsat <- function(data, variables) {
  data %>%
    mutate(EUNISa_1 = as.factor(EUNISa_1)) %>%
    # Remove rows with missing values in predictors
    dplyr::filter(if_all(all_of(variables), ~ !is.na(.))) %>%
    # Remove all rows with wrong values of indices (not between -1 and 1)
    dplyr::filter(NDVI_max <= 1 & NDVI_min >= -1) %>%
    # With Landsat data, many observations removed because of high/low EVI values
    # In Landsat_extraction, values of EVI outside of expected range were set as NA,
    # so better not to use EVI here
    # dplyr::filter(EVI_max <= 1 & EVI_min >= -1) %>%
    dplyr::filter(SAVI_max <= 1 & SAVI_min >= -1) %>%
    dplyr::filter(NDMI_max <= 1 & NDMI_min >= -1) %>%
    dplyr::filter(NDWI_max <= 1 & NDWI_min >= -1) %>%
```

```r
    # Select only variables needed
    dplyr::select(PlotObservationID, EUNISa_1, all_of(variables), Lctnmth,
           valid_1) %>%
    # Keep only rows with differences > 0 in variables
    filter(if_all(any_of(variables[grepl("diff", variables)]),
                  ~ .x > 0)) %>%
    # Select only GPS points
    dplyr::filter(Lctnmth == "Location with GPS" |
                    Lctnmth == "Location with differential GPS")
}
```

```r
# Get filtered data
filtered_data_Landsat <- data_validation_Landsat %>%
  # Rough validation based on NDWI
  rough_validation_Landsat() %>%
  # Data filtering
  filter_data_Landsat(vars_RF_months_Landsat) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value")

# Detect probability columns
prob_cols_Landsat <- names(
  filtered_data_months_Landsat_with_probs_2x
  )[grepl("^prob_", names(filtered_data_months_Landsat_with_probs_2x))]

# calculate auxiliary metrics
df_aux_Landsat_2x <- calculate_df_aux(filtered_data_months_Landsat_with_probs_2x,
                                      prob_cols_Landsat)

# Apply filtering function
df_final_20_10_months_Landsat_2x <- df_aux_Landsat_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)

# Get data for cross-validation
filtered_data_Landsat_cv <- filtered_data_Landsat %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_months_Landsat_2x$PlotObservationID)
```

SatEmb

```r
filter_data_SatEmb <- function(data, variables) {
  data %>%
    mutate(EUNISa_1 = as.factor(EUNISa_1)) %>%
    # Remove rows with missing values in predictors
    dplyr::filter(if_all(all_of(variables), ~ !is.na(.))) %>%
    # Select only variables needed
    dplyr::select(PlotObservationID, EUNISa_1, all_of(variables),
           Lctnmth, valid_1) %>%
    # Select only GPS points
    dplyr::filter(Lctnmth == "Location with GPS" |
                    Lctnmth == "Location with differential GPS")
}
```

```r
data_validation_S2 <- read_tsv(here(
  "data", "clean","final_RS_data_bands_S2_all_20251021.csv"))
```

```
## Rows: 36552 Columns: 137
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr   (7): EUNISa_1, EUNISa_1_descr, EUNISa_2, EUNISa_2_descr, biogeo, unit,...
## dbl (130): PlotObservationID, EVI_auc_mar_oct, EVI_auc_slope, EVI_auc_thresh...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Get filtered data
filtered_data_SatEmb <- data_validation_SatEmb %>%
  # Rough validation based on CH
  rough_validation_SatEmb() %>%
  # Join with S2 data to get NDWI_max info
  left_join(
    data_validation_S2 %>% select(PlotObservationID, NDWI_max)
  ) %>%
  # Rough validation based on NDWI (from S2)
  # Remove those with NDWI_max > 0.3, keep if NDWI_max is NA
  filter(NDWI_max <= 0.3 | is.na(NDWI_max)) %>%
  # Remove unneeded var
  select(-NDWI_max) %>%
  # Data filtering
  filter_data_SatEmb(vars_RF_SatEmb) %>%
  # Join with S2 data to get NDVI_pos_value info
  left_join(
    data_validation_S2 %>% select(PlotObservationID, NDVI_pos_value)
  ) %>%
  # From GPS points, filter based on NDVI percentiles
  filter_by_group_percentiles("NDVI_pos_value") %>%
  # Remove unneeded var
  select(-NDVI_pos_value)
```

```
## Joining with `by = join_by(PlotObservationID)`
## Joining with `by = join_by(PlotObservationID)`
```

```r
# Detect probability columns
prob_cols_SatEmb <- names(
  filtered_data_SatEmb_with_probs_2x
  )[grepl("^prob_", names(filtered_data_SatEmb_with_probs_2x))]

# calculate auxiliary metrics
df_aux_SatEmb_2x <- calculate_df_aux(filtered_data_SatEmb_with_probs_2x,
                                     prob_cols_SatEmb)

# Apply filtering function
df_final_20_10_SatEmb_2x <- df_aux_SatEmb_2x %>%
  group_split(original_EUNIS) %>%
  map_dfr(filter_probs_neg20)
```

```r
# Get data for cross-validation
filtered_data_SatEmb_cv <- filtered_data_SatEmb %>%
  dplyr::filter(PlotObservationID %in% df_final_20_10_SatEmb_2x$PlotObservationID)
```

k-fold cross-validation (4 folds)

```r
set.seed(123)
ctrl <- trainControl(
  method = "cv",
  number = 4, # Creates stratified folds when y is a factor, preserving class proportions
  allowParallel = FALSE,
  savePredictions = "final",
  classProbs = TRUE,     # set TRUE if you also want probabilities (for ROC)
  summaryFunction = multiClassSummary,  # optional, for multi-class summaries
  verboseIter = TRUE      # <--- this gives progress output
)
```

SatEmb

Save

Results

```r
res_SatEmb <- cv_SatEmb$results
res_SatEmb[, c("Accuracy", "Kappa", "AccuracySD", "KappaSD")]
```

Global Accuracy and Kappa

```
##    Accuracy     Kappa  AccuracySD     KappaSD
## 1 0.7590859 0.5812056 0.003656631 0.006504382
```

```r
pred_df_SatEmb <- cv_SatEmb$pred   # todas las predicciones CV
cm_SatEmb <- confusionMatrix(pred_df_SatEmb$pred, pred_df_SatEmb$obs)

by_class_SatEmb <- as.data.frame(cm_SatEmb$byClass)
balanced_acc_SatEmb <- by_class_SatEmb %>%
  mutate(Balanced_Accuracy = (Sensitivity + Specificity)/2) %>%
  select(Sensitivity, Specificity, Balanced_Accuracy)

balanced_acc_SatEmb$Class <- rownames(by_class_SatEmb)
balanced_acc_SatEmb <- balanced_acc_SatEmb %>% select(Class, everything())
balanced_acc_SatEmb
```

Balanced accuracy per class (one-vs-all)

```
##             Class Sensitivity Specificity Balanced_Accuracy
## Class: Q Class: Q   0.6207743   0.8631271         0.7419507
## Class: R Class: R   0.9267099   0.7683087         0.8475093
## Class: S Class: S   0.3303221   0.9585690         0.6444455
## Class: T Class: T   0.9601276   0.9999333         0.9800305
```

```r
balanced_acc_macro_SatEmb <- mean(balanced_acc_SatEmb$Balanced_Accuracy)
balanced_acc_macro_SatEmb
```

Balanced accuracy macro (average over classes)

```
## [1] 0.803484
```

```r
classes_SatEmb <- levels(pred_df_SatEmb$obs)

auc_one_vs_all <- function(df, cls) {
  bin_obs <- as.integer(df$obs == cls)
  prob_cls <- df[[cls]]
  roc_obj <- roc(bin_obs, prob_cls, quiet = TRUE)
  auc(roc_obj)
}

auc_by_class_SatEmb <- sapply(classes_SatEmb, function(cls) auc_one_vs_all(pred_df_SatEmb, cls))
auc_by_class_SatEmb
```

AUC one-vs-all per class + AUC macro

```
##         Q         R         S         T
## 0.8748836 0.9462200 0.8790037 0.9999513
```

```r
auc_macro_SatEmb <- mean(auc_by_class_SatEmb)
auc_macro_SatEmb
```

```
## [1] 0.9250146
```

```r
cm_SatEmb$table
```

Average confusion matrix

```
##           Reference
## Prediction    Q    R    S    T
##          Q 2277  524 1113    0
##          R  968 8143  592   25
##          S  423  119  841    0
##          T    0    1    0  602
```

```r
as.data.frame(cm_SatEmb$byClass)[, c("Precision", "Recall", "F1")]
```

F1-score per class

```
##          Precision    Recall        F1
## Class: Q 0.5817578 0.6207743 0.6006331
## Class: R 0.8370683 0.9267099 0.8796111
## Class: S 0.6080983 0.3303221 0.4280988
## Class: T 0.9983416 0.9601276 0.9788618
```

```r
#varimp_SatEmb <- varImp(cv_SatEmb)
#varimp_SatEmb
```

Variable importance

Save

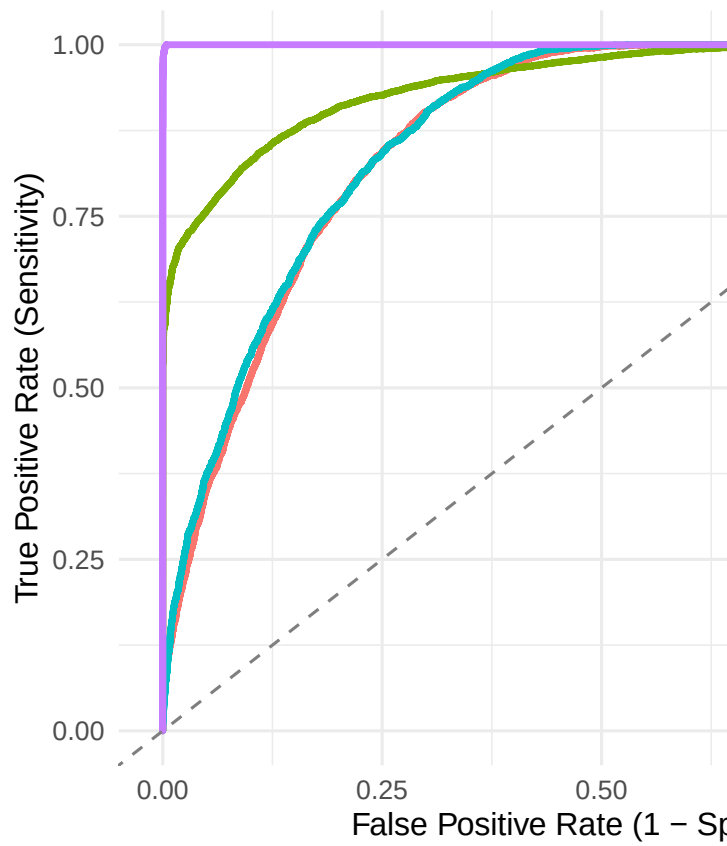
```r
#plot(varimp_SatEmb)
```

Plot

```r
# Compute a ROC object for each class (one-vs-all)
roc_list_SatEmb <- lapply(classes_SatEmb, function(cls) {
  # Binary labels: 1 for cls, 0 for others
  bin_obs <- as.integer(pred_df_SatEmb$obs == cls)
  # Probability for the class
  prob_cls <- pred_df_SatEmb[[cls]]
  # Compute ROC; set direction = "<" if needed depending on your scores
  pROC::roc(response = bin_obs, predictor = prob_cls, quiet = TRUE)
})
names(roc_list_SatEmb) <- classes_SatEmb
```

```r
# Extract ROC curve data (TPR vs FPR) per class
roc_df_SatEmb <- lapply(names(roc_list_SatEmb), function(cls) {
  coords <- pROC::coords(roc_list_SatEmb[[cls]], ret = c("specificity", "sensitivity"), transpose = FALS
  data.frame(Class = cls,
             FPR = 1 - coords$specificity,
             TPR = coords$sensitivity)
}) %>% bind_rows()
```

```r
ggplot(roc_df_SatEmb, aes(x = FPR, y = TPR, color = Class)) +
  geom_line(linewidth = 1.2) +
  #scale_color_manual(values = cols) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "gray50") +
  labs(# title = "ROC curves (one-vs-all) by class",
       x = "False Positive Rate (1 - Specificity)",
       y = "True Positive Rate (Sensitivity)") +
  theme_minimal(base_size = 12)
```



ROC one-vs-all for each class (aggregated over folds)

Landsat

Save

Results

```r
res_Landsat <- cv_Landsat$results
res_Landsat[, c("Accuracy", "Kappa", "AccuracySD", "KappaSD")]
```

Global Accuracy and Kappa

```
##    Accuracy    Kappa  AccuracySD    KappaSD
## 1 0.7473844 0.570208 0.007142994 0.01299143
```

```r
pred_df_Landsat <- cv_Landsat$pred   # todas las predicciones CV
cm_Landsat <- confusionMatrix(pred_df_Landsat$pred, pred_df_Landsat$obs)

by_class_Landsat <- as.data.frame(cm_Landsat$byClass)
balanced_acc_Landsat <- by_class_Landsat %>%
  mutate(Balanced_Accuracy = (Sensitivity + Specificity)/2) %>%
  select(Sensitivity, Specificity, Balanced_Accuracy)

balanced_acc_Landsat$Class <- rownames(by_class_Landsat)
balanced_acc_Landsat <- balanced_acc_Landsat %>% select(Class, everything())
balanced_acc_Landsat
```

Balanced accuracy per class (one-vs-all)

```
##             Class Sensitivity Specificity Balanced_Accuracy
## Class: Q Class: Q   0.6392080   0.8466412         0.7429246
## Class: R Class: R   0.9206519   0.8035668         0.8621093
## Class: S Class: S   0.3024079   0.9436193         0.6230136
## Class: T Class: T   0.9519112   0.9973900         0.9746506
```

```r
balanced_acc_macro_Landsat <- mean(balanced_acc_Landsat$Balanced_Accuracy)
balanced_acc_macro_Landsat
```

Balanced accuracy macro (average over classes)

```
## [1] 0.8006745
```

```r
classes_Landsat <- levels(pred_df_Landsat$obs)

auc_one_vs_all <- function(df, cls) {
  bin_obs <- as.integer(df$obs == cls)
  prob_cls <- df[[cls]]
  roc_obj <- roc(bin_obs, prob_cls, quiet = TRUE)
  auc(roc_obj)
}

auc_by_class_Landsat <- sapply(classes_Landsat, function(cls) auc_one_vs_all(pred_df_Landsat, cls))
auc_by_class_Landsat
```

AUC one-vs-all per class + AUC macro

```
##         Q         R         S         T
## 0.8751250 0.9565329 0.8660378 0.9996130
```

```r
auc_macro_Landsat <- mean(auc_by_class_Landsat)
auc_macro_Landsat
```

```
## [1] 0.9243272
```

```r
cm_Landsat$table
```

Average confusion matrix

```
##           Reference
## Prediction     Q     R     S     T
##          Q  4229   864  2050    15
##          R  1377 12937   892    22
##          S   982   227  1281     2
##          T    28    24    13   772
```

```r
as.data.frame(cm_Landsat$byClass)[, c("Precision", "Recall", "F1")]
```

F1-score per class

```
##          Precision    Recall        F1
## Class: Q 0.5908075 0.6392080 0.6140555
## Class: R 0.8495535 0.9206519 0.8836749
## Class: S 0.5140449 0.3024079 0.3807967
## Class: T 0.9223417 0.9519112 0.9368932
```

```r
#varimp_Landsat <- varImp(cv_Landsat)
#varimp_Landsat
```

Variable importance

Save

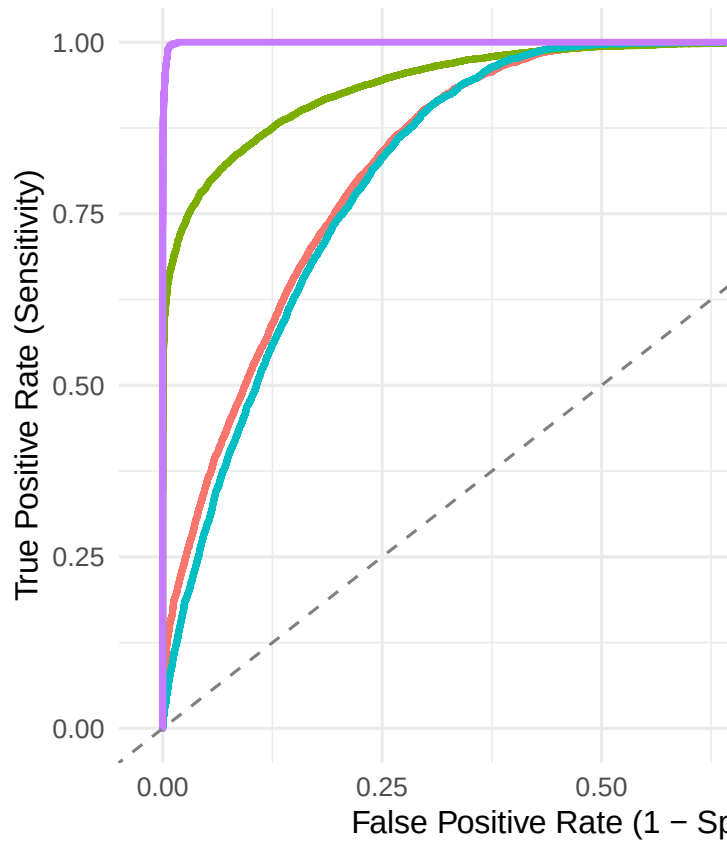
```r
#plot(varimp_Landsat)
```

Plot

```r
# Compute a ROC object for each class (one-vs-all)
roc_list_Landsat <- lapply(classes_Landsat, function(cls) {
  # Binary labels: 1 for cls, 0 for others
  bin_obs <- as.integer(pred_df_Landsat$obs == cls)
  # Probability for the class
  prob_cls <- pred_df_Landsat[[cls]]
  # Compute ROC; set direction = "<" if needed depending on your scores
  pROC::roc(response = bin_obs, predictor = prob_cls, quiet = TRUE)
})
names(roc_list_Landsat) <- classes_Landsat
```

```r
# Extract ROC curve data (TPR vs FPR) per class
roc_df_Landsat <- lapply(names(roc_list_Landsat), function(cls) {
  coords <- pROC::coords(roc_list_Landsat[[cls]], ret = c("specificity", "sensitivity"), transpose = FA
  data.frame(Class = cls,
             FPR = 1 - coords$specificity,
             TPR = coords$sensitivity)
}) %>% bind_rows()

ggplot(roc_df_Landsat, aes(x = FPR, y = TPR, color = Class)) +
  geom_line(linewidth = 1.2) +
  #scale_color_manual(values = cols) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "gray50") +
  labs(# title = "ROC curves (one-vs-all) by class",
       x = "False Positive Rate (1 - Specificity)",
       y = "True Positive Rate (Sensitivity)") +
  theme_minimal(base_size = 12)
```



ROC one-vs-all for each class (aggregated over folds)

Session info

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Spain.utf8  LC_CTYPE=Spanish_Spain.utf8
## [3] LC_MONETARY=Spanish_Spain.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Spain.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
##  [1] stats4    grid      parallel  stats     graphics  grDevices utils
##  [8] datasets  methods   base
##
```

```
## other attached packages:
##  [1] MLmetrics_1.1.3      permimp_1.1-0        beepr_2.0
##  [4] rlang_1.1.6          pROC_1.19.0.1        randomForest_4.7-1.2
##  [7] caret_7.0-1          lattice_0.22-7       moreparty_0.4.2
## [10] party_1.3-18         strucchange_1.5-4    sandwich_3.1-1
## [13] zoo_1.8-14           modeltools_0.2-24    mvtnorm_1.3-3
## [16] doParallel_1.0.17    iterators_1.0.14     foreach_1.5.2
## [19] here_1.0.2           lubridate_1.9.4      forcats_1.0.1
## [22] stringr_1.6.0        dplyr_1.1.4          purrr_1.2.0
## [25] readr_2.1.6          tidyr_1.3.1          tibble_3.3.0
## [28] ggplot2_4.0.1        tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] magrittr_2.0.3       multcomp_1.4-29      otel_0.2.0
##  [4] e1071_1.7-16         matrixStats_1.5.0    compiler_4.5.2
##  [7] reshape2_1.4.5       vctrs_0.6.5          crayon_1.5.3
## [10] pkgconfig_2.0.3      fastmap_1.2.0        labeling_0.4.3
## [13] promises_1.5.0       phosphoricons_0.2.1  rmarkdown_2.30
## [16] prodlim_2025.04.28   tzdb_0.5.0           bit_4.6.0
## [19] xfun_0.54            cachem_1.1.0         jsonlite_2.0.0
## [22] recipes_1.3.1        later_1.4.4          R6_2.6.1
## [25] coin_1.4-3           bslib_0.9.0          stringi_1.8.7
## [28] RColorBrewer_1.1-3   parallelly_1.45.1    rpart_4.1.24
## [31] jquerylib_0.1.4      Rcpp_1.1.0           knitr_1.50
## [34] future.apply_1.20.1  rclipboard_0.2.1     audio_0.1-11
## [37] httpuv_1.6.16        Matrix_1.7-4         splines_4.5.2
## [40] nnet_7.3-20          varImp_0.4           timechange_0.3.0
## [43] tidyselect_1.2.1     rstudioapi_0.17.1    yaml_2.3.10
## [46] timeDate_4051.111    codetools_0.2-20     listenv_0.10.0
## [49] plyr_1.8.9           shiny_1.12.1         withr_3.0.2
## [52] S7_0.2.1             evaluate_1.0.5       future_1.68.0
## [55] survival_3.8-3       proxy_0.4-27         pillar_1.11.1
## [58] DT_0.34.0            generics_0.1.4       vroom_1.6.7
## [61] rprojroot_2.1.1      hms_1.1.4            scales_1.4.0
## [64] globals_0.18.0       xtable_1.8-4         class_7.3-23
## [67] glue_1.8.0           tools_4.5.2          data.table_1.17.8
## [70] ModelMetrics_1.2.2.2 gower_1.0.2          measures_0.3
## [73] libcoin_1.0-10       ipred_0.9-15         nlme_3.1-168
## [76] cli_3.6.5            lava_1.8.2           gtable_0.3.6
## [79] sass_0.4.10          digest_0.6.39        TH.data_1.1-5
## [82] htmlwidgets_1.6.4    farver_2.1.2         htmltools_0.5.9
## [85] lifecycle_1.0.4      hardhat_1.4.2        shinyWidgets_0.9.0
## [88] mime_0.13            bit64_4.6.0-1        MASS_7.3-65
```